输入标题

现有功能模块：

- 用户系统（登录、个人资料、权限管理）
- 内容管理（党建新闻、活动、学习资料）
- 团课学习系统
- 答题系统（单选、多选、判断题）
- 留言系统
- AI 问答（已接入 DeepSeek API）
- 搜索功能
- 通知系统
- 学习记录和收藏
- 文件上传下载

当前数据存储： 全部使用 localStorage，需要迁移到后端数据库

🏗️ 推荐架构设计

1. 技术栈建议

前端：

- 保持当前 HTML/CSS/JavaScript 架构
- 或升级为 Vue.js/React 框架（推荐 Vue.js）

后端：

- Node.js + Express （与前端 JavaScript 统一）
- 或 Python + Flask/Django
- 或 Java + Spring Boot

数据库：

- MySQL （关系型数据库，适合结构化数据）
- MongoDB （文档型数据库，适合灵活数据）
- Redis （缓存、会话管理）

其他服务：

- Nginx （反向代理、静态文件服务）
- Docker （容器化部署）

2. 项目目录结构

Plain Text

```
zhihui-party/
├── frontend/                 # 前端项目
│   ├── public/
│   │   └── index.html       # 主页面
│   ├── src/
│   │   ├── assets/          # 静态资源
│   │   ├── css/             # 样式文件
│   │   ├── js/              # JavaScript文件
│   │   │   ├── api.js       # API调用封装
│   │   │   ├── auth.js      # 认证相关
│   │   │   ├── utils.js     # 工具函数
│   │   │   └── modules/     # 功能模块
│   │   └── config.js        # 配置文件
│   └── package.json
│
├── backend/                  # 后端项目
│   ├── src/
│   │   ├── controllers/     # 控制器
│   │   │   ├── auth.js      # 认证控制器
│   │   │   ├── user.js      # 用户控制器
│   │   │   ├── content.js   # 内容控制器
│   │   │   ├── quiz.js      # 答题控制器
│   │   │   ├── course.js    # 课程控制器
│   │   │   ├── message.js   # 留言控制器
│   │   │   └── search.js    # 搜索控制器
│   │   ├── models/          # 数据模型
│   │   │   ├── User.js
│   │   │   ├── Content.js
│   │   │   ├── Quiz.js
│   │   │   ├── Course.js
│   │   │   └── Message.js
│   │   ├── routes/          # 路由
│   │   ├── middleware/      # 中间件
│   │   │   ├── auth.js      # 认证中间件
│   │   │   ├── admin.js     # 管理员中间件
│   │   │   └── upload.js    # 文件上传中间件
│   │   ├── services/        # 业务逻辑
│   │   ├── config/          # 配置
│   │   │   ├── database.js  # 数据库配置
│   │   │   └── ai.js        # AI配置
│   │   └── utils/           # 工具函数
│   ├── uploads/             # 上传文件存储
```

```
│   ├── package.json
│   └── .env                    # 环境变量
│
├── database/                    # 数据库相关
│   ├── migrations/             # 数据库迁移
│   └── seeds/                  # 初始数据
│
├── nginx/                      # Nginx配置
│   └── nginx.conf
│
└── docker-compose.yml          # Docker编排
```

3. 需要的后端 API 接口

🔐 认证相关

Plain Text

```
POST   /api/auth/register       # 用户注册
POST   /api/auth/login          # 用户登录
POST   /api/auth/logout         # 用户登出
POST   /api/auth/refresh        # 刷新token
GET    /api/auth/me             # 获取当前用户信息
```

👤 用户相关

Plain Text

```
GET    /api/users/profile       # 获取用户资料
PUT    /api/users/profile       # 更新用户资料
GET    /api/users/progress      # 获取学习进度
GET    /api/users/records       # 获取学习记录
GET    /api/users/collections   # 获取收藏列表
POST   /api/users/collections   # 添加收藏
DELETE /api/users/collections/:id  # 删除收藏
GET    /api/users/wrong-questions  # 获取错题本
```

📰 内容管理

Plain Text

```
GET    /api/content              # 获取内容列表
GET    /api/content/:id          # 获取内容详情
POST   /api/content              # 创建内容（管理员）
PUT    /api/content/:id          # 更新内容（管理员）
DELETE /api/content/:id          # 删除内容（管理员）
GET    /api/content/category/:category  # 按分类获取内容
```

📚 团课学习

Plain Text

```
GET    /api/courses              # 获取课程列表
GET    /api/courses/:id          # 获取课程详情
GET    /api/courses/:id/chapters  # 获取课程章节
POST   /api/courses/:id/progress  # 更新学习进度
```

📝 答题系统

Plain Text

```
GET    /api/quiz                 # 获取题库
GET    /api/quiz/random          # 随机获取题目
POST   /api/quiz/submit          # 提交答案
GET    /api/quiz/results         # 获取答题结果
POST   /api/quiz/import          # 导入题库（管理员）
DELETE /api/quiz/:id             # 删除题目（管理员）
```

💬 留言系统

Plain Text

```
GET    /api/messages             # 获取留言列表
POST   /api/messages             # 提交留言
PUT    /api/messages/:id/approve  # 审核留言（管理员）
DELETE /api/messages/:id         # 删除留言（管理员）
```

🔍 搜索功能

```
GET    /api/search              # 搜索内容
GET    /api/search/history      # 获取搜索历史
DELETE /api/search/history      # 清除搜索历史
```

📢 通知系统

```
GET    /api/notifications       # 获取通知列表
PUT    /api/notifications/:id/read  # 标记已读
POST   /api/notifications       # 发送通知（管理员）
```

📁 文件管理

```
POST   /api/upload              # 上传文件
GET    /api/download/:id        # 下载文件
DELETE /api/files/:id           # 删除文件（管理员）
```

🤖 AI 问答

```
POST   /api/ai/chat             # AI对话接口
```

4. 数据库设计

用户表 (users)

```sql
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  name VARCHAR(50),
  identity VARCHAR(20),
  major VARCHAR(50),
  position VARCHAR(50),
  contact VARCHAR(50),
  role ENUM('user', 'admin') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

内容表 (content)

```sql
CREATE TABLE content (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(200) NOT NULL,
  category VARCHAR(50) NOT NULL,
  content TEXT,
  date DATE,
  author VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

课程表 (courses)

SQL

```sql
CREATE TABLE courses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(200) NOT NULL,
  description TEXT,
  cover_image VARCHAR(255),
  total_chapters INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

学习进度表 (course_progress)

SQL

```sql
CREATE TABLE course_progress (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  course_id INT,
  completed_chapters INT DEFAULT 0,
  progress_percentage INT DEFAULT 0,
  last_study_time TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (course_id) REFERENCES courses(id),
  UNIQUE KEY (user_id, course_id)
);
```

题库表 (quiz)

SQL

```sql
CREATE TABLE quiz (
  id INT PRIMARY KEY AUTO_INCREMENT,
  question TEXT NOT NULL,
  type ENUM('single', 'multiple', 'judgment') NOT NULL,
  options JSON,
  answer JSON NOT NULL,
  explanation TEXT,
  difficulty ENUM('easy', 'medium', 'hard') DEFAULT 'medium',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

答题记录表 (quiz_records)

```sql
CREATE TABLE quiz_records (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  quiz_id INT,
  user_answer JSON,
  is_correct BOOLEAN,
  answer_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (quiz_id) REFERENCES quiz(id)
);
```

留言表 (messages)

```sql
CREATE TABLE messages (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT,
  content TEXT NOT NULL,
  status ENUM('pending', 'approved', 'rejected') DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

```

收藏表 (collections)

```sql
CREATE TABLE collections (
id INT PRIMARY KEY AUTO_INCREMENT,
user_id INT,
content_id INT,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id),
FOREIGN KEY (content_id) REFERENCES content(id),
UNIQUE KEY (user_id, content_id)
);
```

5. 前端改造要点

5.1 创建 API 调用封装

JavaScript

```javascript
// src/js/api.js
const API_BASE_URL = 'http://localhost:3000/api';

const api = {
  // 认证
  login: (data) => fetch(`${API_BASE_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  }),

  // 内容
  getContent: (id) => fetch(`${API_BASE_URL}/content/${id}`),

  // 答题
  getQuiz: () => fetch(`${API_BASE_URL}/quiz`),
  submitQuiz: (data) => fetch(`${API_BASE_URL}/quiz/submit`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  }),

  // 其他API...
};

export default api;
```

5.2 修改 localStorage 调用为 API 调用

JavaScript

```javascript
// 修改前
const profile = JSON.parse(localStorage.getItem('userProfile'));

// 修改后
const response = await api.getUserProfile();
const profile = await response.json();
```

5.3 添加认证中间件

JavaScript

```javascript
// src/js/auth.js
const token = localStorage.getItem('token');

const authHeaders = {
  'Authorization': `Bearer ${token}`,
  'Content-Type': 'application/json'
};
```

6. 后端核心代码示例

6.1 Express 服务器设置

JavaScript

```javascript
// backend/src/app.js
const express = require('express');
const cors = require('cors');
const bodyParser = require('body-parser');
const authRoutes = require('./routes/auth');
const userRoutes = require('./routes/user');
const contentRoutes = require('./routes/content');

const app = express();

app.use(cors());
app.use(bodyParser.json());
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/content', contentRoutes);

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

6.2 数据库连接

JavaScript

```javascript
// backend/src/config/database.js
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

7. 部署方案

7.1 Docker 部署

YAML

```yaml
# docker-compose.yml
version: '3.8'
services:
  frontend:
    build: ./frontend
    ports:
      - "80:80"

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=mysql
      - DB_USER=root
      - DB_PASSWORD=password
      - DB_NAME=zhihui_party
    depends_on:
      - mysql

  mysql:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=password
      - MYSQL_DATABASE=zhihui_party
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

8. 开发步骤建议

1. 第一阶段：基础架构
   - 搭建后端服务器
   - 配置数据库连接
   - 实现用户认证系统
2. 第二阶段：核心功能
   - 内容管理 API

    - 答题系统 API
    - 学习记录 API
3. 第三阶段：前端改造
    - 创建 API 调用封装
    - 逐步替换 localStorage 为 API 调用
    - 添加错误处理和加载状态
4. 第四阶段：优化部署
    - Docker 容器化
    - Nginx 配置
    - 生产环境部署

9. 安全考虑

- 使用 JWT 进行身份认证
- 密码加密存储（bcrypt）
- SQL 注入防护
- XSS 防护
- CORS 配置
- 文件上传安全验证
- API 限流